

Ropo Educational Robot — System Documentation

Graduation Project — Senior Software Architecture

AI-Powered Interactive Educational Robot with Bilingual Voice & Vision Pipelines

Table of Contents

- 1. Project Overview
- 2. Architecture Overview
- 3. Files Overview Table
- 4. Detailed File Explanations
- 5. Code Walkthrough
- 6. Sequence Flow
- 7. Folder Integration
- 8. Technical Notes
- 9. Academic Explanation

1. Project Overview

Ropo is an interactive bilingual (Arabic/English) educational robot that combines a voice conversation pipeline, real-time computer vision, an animated facial display, and motor control into a single cohesive system. Designed to run on a **Raspberry Pi 4/5** or a **Windows desktop** during development, Ropo serves as a platform for classroom interactions, STEM education, and human–robot interaction research.

Core Features

- **Voice Pipeline** — Silero VAD + Google ASR + OpenRouter LLM + edge-tts TTS in a real-time streaming architecture

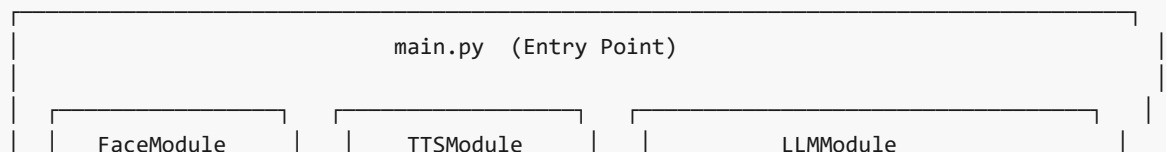
- **Vision Pipeline** — Face tracking (OpenCV DNN), gesture recognition (MediaPipe), emotion detection (CNN), object detection (YOLOv8n), obstacle detection, scene segmentation (YOLOv8n-seg)
- **Animated Face** — Pygame-based display with five states: IDLE, LISTENING, THINKING, SPEAKING, CURIOUS
- **Motor Control** — Serial communication with ESP32/Arduino for physical movement (forward, backward, turn, stop)
- **Session Memory** — SQLite-backed conversation history with sliding-window + summarization for context
- **Adaptive Load Shedding** — Runtime vision-profile switching (minimal / balanced / full) for thermal management
- **Remote Shutdown** — Web API polling client for safe Raspberry Pi shutdown from a mobile app backend
- **Diagnostic Suite** — Comprehensive health check covering platform, packages, model files, network, live module tests, and integration

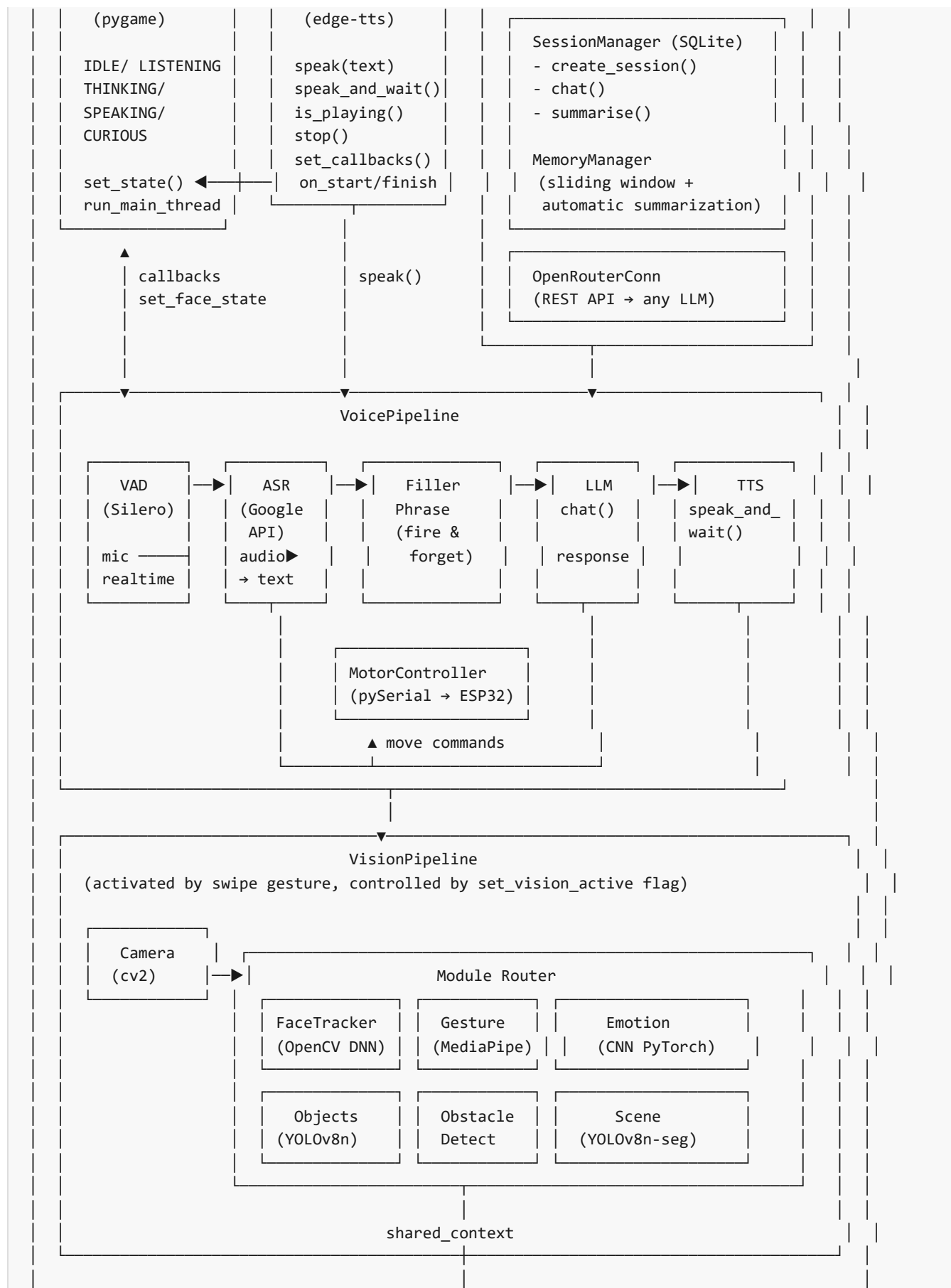
Architecture Summary

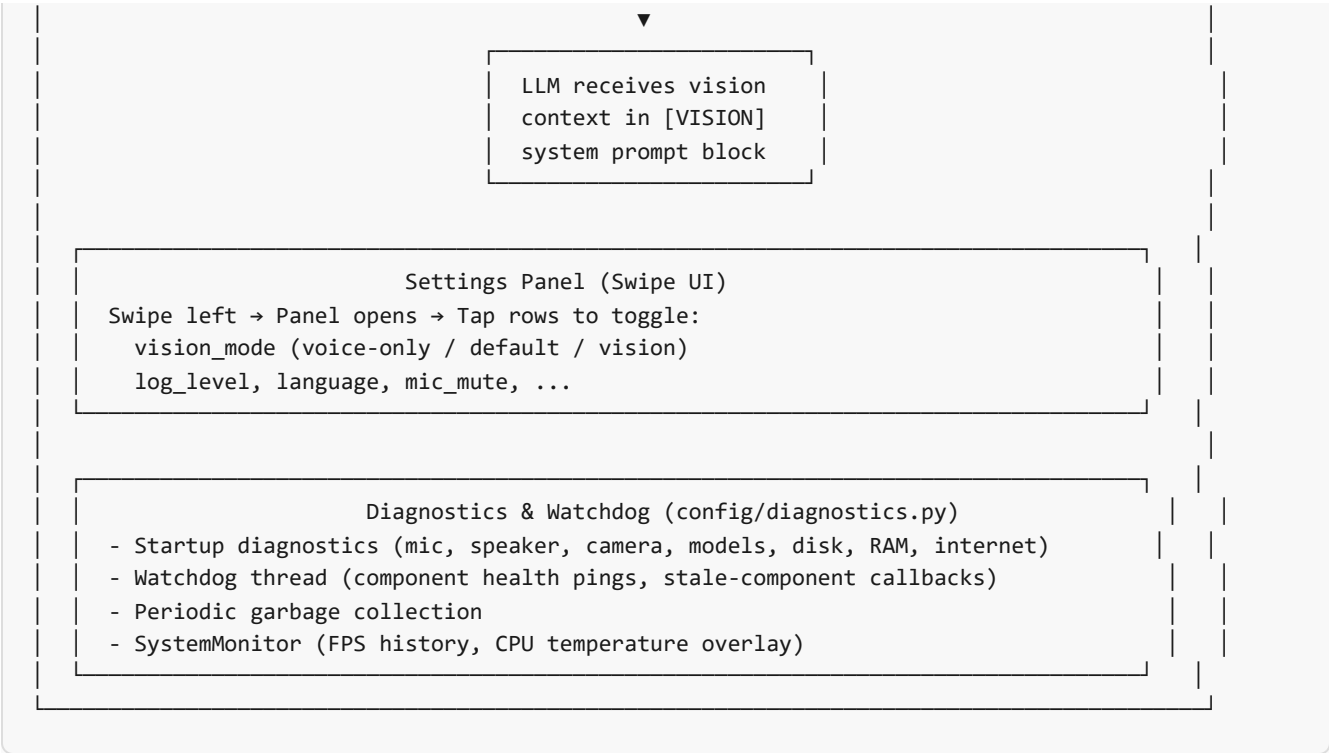
Ropo follows a **modular pipeline architecture** orchestrated by a single entry point (`main.py`). Each subsystem (voice, vision, LLM, face, hardware) is an independent module communicating via **dependency injection** and **callback-based observation**. The voice pipeline runs on two threads (audio capture + segment processing); the vision pipeline runs on its own thread; and the Pygame face runs on the main thread. Modules share context through a **shared-context dictionary** pattern rather than direct coupling.

2. Architecture Overview

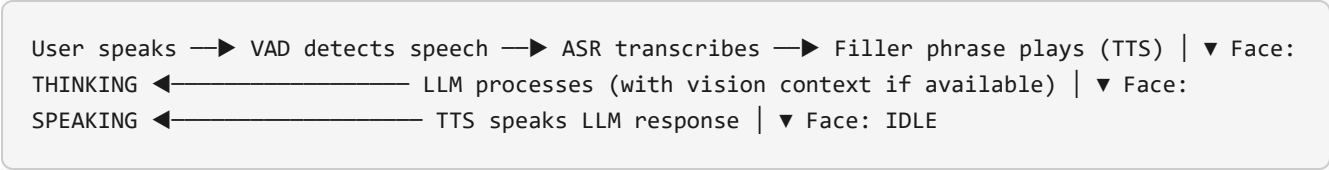
The following diagram illustrates every module initialized by `main.py` and the directional data flow between them.







Data Flow (Single Voice Interaction Cycle)



3. Files Overview Table

File	Lines	Role	Key Responsibilities
main.py	141	Entry point	Module initialization, dependency injection, main loop, graceful shutdown
health_check.py	404	Diagnostics	ANSI-coloured standalone health check with live module tests and readiness scoring
test_vision_debug.py	218	Vision debug tool	Captures 50 frames through all vision modules; saves annotated images; per-module timing

File	Lines	Role	Key Responsibilities
robot_shutdown_client.py	72	RPi shutdown poller	Polls a REST API every 15 s; executes <code>sudo shutdown -h now</code> on signal
requirements.txt	24	Dependencies	Pinned Python packages for core, voice, vision, face, and shutdown API modules
.env.example	89	Config template	Bilingual (Arabic/English) environment variable documentation with defaults
setup.sh	204	Linux/RPi setup	Automates venv creation, system packages, model downloads, Pi optimizations, health check
setup.ps1	167	Windows setup	PowerShell equivalent: venv, pip install, directory creation, .env copy, model downloads
ropo-shutdown.service	15	systemd unit	Runs robot_shutdown_client.py as a system service with auto-restart
README.md	373	Project documentation	Feature list, architecture diagram, setup instructions, roadmap, team contribution guide

4. Detailed File Explanations

4.1 main.py — Entry Point & System Orchestrator

`main.py` is the single entry point for the entire robot system. Its startup sequence is divided into six clear phases:

- Environment & Logging Setup** — Loads `.env` via `python-dotenv`, detects the platform preset (desktop/RPi4/RPi5), then calls `setup_logging()` which creates a `RotatingFileHandler` (5 MB max, 3 backups) to `data/robot.log` plus a custom `AsciiHandler` that replaces non-ASCII characters with `?` for safe console output. Also sets `ultraalytics` logger to `WARNING` to reduce noise.
- Platform Configuration** — On RPi, calls `configure_platform()` which limits OpenCV and PyTorch to single-threaded mode for thermal efficiency.

3. **Startup Diagnostics** — Runs `run_startup_diagnostics()` checking microphone, speaker, camera, model files, disk space, RAM, and internet connectivity. Results are printed to console.
4. **Module Initialization** — Creates instances in dependency order:
 - `FaceModule(fullscreen=False)` — Pygame face display; started immediately so the face is visible during boot
 - `TTSModule(settings.tts)` — edge-tts audio playback engine
 - `LLModule()` — OpenRouter client + SQLite SessionManager
 - `session_id` — A conversation session is created with `llm.create_session(student_name, language)`
 - `VisionPipeline()` — Camera + all vision modules; opened and started on its own thread
 - `MotorController(port)` — Serial connection to ESP32; gracefully degrades if unavailable
 - `VoicePipeline(...)` — Receives all modules via dependency injection, plus a `set_face_state` callback and `vision_context_getter` for cross-pipeline data sharing
5. **Main Loop** — Calls `face.run_main_thread()` which enters the Pygame event loop and blocks. All other modules run on their own threads.
6. **Shutdown Sequence** — In the `finally` block: stops the voice pipeline (joins stream + worker threads), stops and closes the vision pipeline, closes the motor serial port, closes the LLM database, and stops the face display.

Cross-platform Port Detection

Motor port is selected at runtime:

```
if sys.platform == "win32":
    motor_port = "COM3"
else:
    motor_port = "/dev/ttyS0"
```

4.2 `health_check.py` — Enhanced Standalone Diagnostics

A 404-line diagnostic script that can be run independently before the main system. It produces both ANSI-coloured terminal output and a structured `health_report.txt` file. The diagnostic categories are:

Category	Checks Performed	Methodology
Platform Info	OS, Python version, CPU cores, RAM, RPi detection, working directory	<code>platform.system()</code> , <code>psutil</code> , <code>/proc/device-tree/model</code>
Package Versions	numpy, requests, dotenv, speechrecognition, sounddevice, torch, edge-tts, cv2, ultralytics, torchvision, pygame	<code>importlib.metadata.version()</code>
Model Files	deploy.prototxt, res10_300x300_ssd, yolov8s.pt, yolov8s-seg.pt, emotion_cnn_pytorch.pt	File existence + size check
VAD Settings	ROBOT_VAD_THRESHOLD, SILENCE_TIMEOUT, MIN_SPEECH, PRE_ROLL	<code>os.getenv()</code> vs defaults
Network	OpenRouter API (api.openrouter.ai:443), Google ASR (www.google.com:443)	<code>socket.connect()</code> with 5 s timeout
Module Imports	All 17 internal modules across voice, vision, llm, and config packages	<code>importlib.import_module()</code> with exception capture
Live Module Tests	VAD inference, TTS generation, LLM API call, Camera capture	Actual function calls with timing; LLM test requires API key
Integration Tests	FaceModule, TTSMModule, LLMModule, VisionPipeline, VoicePipeline instantiation	Constructor calls with minimal arguments

Readiness Scoring

After all checks, the system computes a score: `passed / total * 100`. Thresholds: $\geq 90\%$ → READY, $\geq 70\%$ → MOSTLY READY, otherwise → NOT READY. The score is printed in green/yellow/red ANSI and saved to the report.

4.3 `test_vision_debug.py` — Standalone Vision Debug Tool

This tool exercises every vision module in isolation from the main system. It captures 50 frames from the camera and runs each module on every frame, logging per-module timing. Annotated frames are saved every 10 frames to `vision_debug_output/`.

Debug workflow:

1. Opens camera via `CameraManager`
2. Initializes all modules: `FacelIdentityTracker`, `GestureDetector`, `EmotionDetector`, `ObjectRecognitionModule`, `SceneSegmentationModule`, `ObstacleDetector`
3. For each frame: runs all modules sequentially, records timing, draws bounding boxes and labels
4. Logs per-frame summary: faces count, objects count, gesture status, and six timing values
5. Saves annotated JPGs with overlay info (frame number, face/gesture/emotion/objects, milliseconds)
6. Prints final summary with detection counts and warnings if no faces or objects were found

4.4 `robot_shutdown_client.py` — Raspberry Pi Shutdown Poller

A lightweight, resilient polling client designed to run as a systemd service on the Raspberry Pi. It enables remote shutdown from a mobile app or web backend.

Polling architecture:

- Reads `SHUTDOWN_API_URL` and `SHUTDOWN_TOKEN` from environment variables
- Uses a single `requests.Session()` with a 10-second timeout
- Polls `GET /shutdown-status` every 15 seconds (configurable via `SHUTDOWN_POLL_INTERVAL`)
- When the JSON response contains `"shutdown": true` :
 1. Sends `POST /shutdown-reset` with the token to acknowledge and reset the flag
 2. Executes `sudo shutdown -h now` via subprocess
- Never crashes: all exceptions are caught and logged, and the loop continues indefinitely
- Individual `except` clauses for `Timeout`, `ConnectionError`, generic `RequestException`, and any unexpected error

4.5 Configuration Files

`.env.example`

All 35+ environment variables are documented bilingually (Arabic + English) with sensible defaults. Categories: LLM settings (API key, model, timeouts), general (log level, student name, language), VAD (threshold, silence timeout, chunk size, model repo), ASR (provider, sample rate, language mode), camera (resolution, FPS, face detection parameters), vision (profile, overlay, adaptive shedding), TTS (engine, poll interval), gesture (buffer, stability, confidence), and remote shutdown (API URL, token).

`requirements.txt`

Twenty-four lines organized into six logical groups: Core (numpy, pyserial, requests, dotenv), Shutdown API (fastapi, uvicorn — deployed separately on Railway), Voice Pipeline (rapidfuzz, speechrecognition, sounddevice, torch, edge-tts), Vision Pipeline (opencv-python, ultralytics, torchvision), and Face Display (pygame).

4.6 Deployment Scripts

`setup.sh` (Linux / Raspberry Pi)

An idempotent 204-line bash script that: checks Python ≥ 3.10 , installs system packages (portaudio, SDL2, OpenBLAS, OpenCV, ffmpeg, libcamera), creates a Python venv and installs requirements, creates project directories with `.gitkeep`, copies `.env.example` to `.env` if missing, downloads OpenCV face detection models from GitHub, applies Pi-specific optimizations (camera interface, `gpu_mem=128`, `picamera2`), and runs the health check as a final validation.

`setup.ps1` (Windows)

A 167-line PowerShell 5.1+ equivalent. It performs the same steps as `setup.sh` but uses `Test-Path`, `Copy-Item`, `Invoke-WebRequest`, and PowerShell colour functions. Does not include Pi-specific optimizations.

`ropo-shutdown.service`

A systemd unit file that runs `robot_shutdown_client.py` as a service. Configured with `Restart=always` (5-second delay), `EnvironmentFile=/etc/ropo-shutdown.env`, and `Wants=network-online.target` to ensure network availability before starting.

5. Code Walkthrough

The following screenshots illustrate the key sections of the codebase. Each placeholder represents a figure that should be captured from the actual source files.

Figure 5.1 — `main.py` startup sequence: environment loading, preset detection, and call to `setup_logging()`. Lines 1–23 show the imports and global settings initialization.

Figure 5.2 — `setup_logging()` (lines 25–57): `RotatingFileHandler` creation with 5 MB max size and 3 backups; custom `AsciiHandler` class that encodes → replaces non-ASCII → decodes for safe console output; dual-handler logger configuration.

Figure 5.3 — `main()` module initialization block (lines 75–101): `FaceModule`, `TTSModule`, `LLMModule`, session creation, `VisionPipeline`, `MotorController` with platform-specific port detection, and the `set_face_state` callback closure.

Figure 5.4 — `VoicePipeline` instantiation with dependency injection (lines 108–115): all five dependencies (`llm`, `tts_module`, `session_id`, `face_set_state`, `motor_controller`, `vision_context_getter`) passed explicitly, demonstrating the dependency-injection pattern.

Figure 5.5 — Cleanup and shutdown sequence (lines 123–141): the `try/except/finally` block showing the ordered teardown of all six modules with logging at each step.

Figure 5.6 — `health_check.py` diagnostics loop pattern (lines 41–44, 205–250): the `check()` helper that appends results and the live VAD inference test that loads Silero, runs inference on a zero array, and validates the output.

Figure 5.7 — `test_vision_debug.py` multi-module testing (lines 93–212): the main capture loop that runs six vision modules per frame, logs per-module timing, and conditionally saves annotated images with bounding boxes and overlay information.

Figure 5.8 — `VoicePipeline.run_forever()` VAD callback (lines 337–387): the `sounddevice.InputStream` callback demonstrating the state-machine logic for speech detection, buffering, pre-roll, silence timeout, and segment enqueueing.

Figure 5.9 — `robot_shutdown_client.py` polling loop (lines 31–68): the complete `poll_forever()` function showing the GET-then-POST pattern, error-resilient design with six separate exception handlers, and subprocess-based shutdown execution.

Figure 5.10 — `config/settings.py` data model (lines 18–197): the frozen dataclass hierarchy with `get_settings()` factory function, `PlatformPreset` enum, `detect_preset()` platform detection, `configure_platform()` RPi optimizations, and `profile_module_config()` for vision profile mapping.

6. Sequence Flow

The following sequence diagram illustrates the complete lifecycle from system startup through idle operation, a voice interaction, and shutdown.

```

SYSTEM STARTUP ===== main.py | | load_dotenv() # .env → os.environ | | detect_preset()
# DESKTOP_DEBUG / RPi4 / RPi5 | | setup_logging() # RotatingFileHandler + AsciiHandler | |
configure_platform() # (RPi) cv2/torch single-thread | | run_startup_diagnostics() # mic,
speaker, camera, models, ... | | FaceModule() —.start() # pygame window opens (IDLE state) | |
TTSModule(settings.tts) # edge-tts engine | | LLMModule() # OpenRouter + SQLite | |
create_session(student, lang) # → session_id | | VisionPipeline() —.open() —.start() # camera
+ vision thread | | MotorController(port) # serial to ESP32 | | VoicePipeline(llm, tts, face_cb,
motor, vision_ctx) | | .start() # spawns stream + worker threads | | face.run_main_thread() #
[BLOCKING] pygame event loop IDLE STATE ===== Face: IDLE VoicePipeline: stream thread
listening on mic VisionPipeline: frames processed per profile config (minimal/balanced/full)
Watchdog: pinging components every 30 s SystemMonitor: tracking FPS / temperature VOICE
INTERACTION ===== 1. User speaks 2. VAD callback detects speech (is_speech() ==
True) | | Interrupts current TTS if playing | | Sets FaceState LISTENING | | Begins accumulating
audio chunks | | Pre-roll buffer prepended 3. Silence timeout reached (≥1.2 s no speech) | |
Segment enqueued to worker thread 4. Worker: a. ASR transcribes (Google API) → text + language
b. Wake-word check (fuzzy match) c. If wake word: speak greeting, return d. Move-command check
(fuzzy match against MOVE_COMMANDS) → MotorController.
{forward,backward,turn_left,turn_right,stop}() e. Face: THINKING f. Filler phrase plays via TTS
(fire-and-forget) g. LLM.chat(session_id, text, vision_context) | | If vision data available:
prepend [VISION] context block | | If OpenRouter unreachable: offline fallback message h. Face:
SPEAKING i. TTS.speak_and_wait(response, language) j. Face: IDLE 5. Return to idle SHUTDOWN
===== KeyboardInterrupt or Exception | | finally: | | voice_pipeline.stop() # set
_running=False, join threads | | vision_pipeline.stop() | | vision_pipeline.close() | |
motor.close() # close serial port | | llm.close() # close SQLite connection | | face.stop() #
close pygame

```

7. Folder Integration

All six subdirectories are connected exclusively through `main.py`. No module imports another module from a different vertical subsystem — cross-vertical communication happens via the `VoicePipeline` constructor parameters (dependency injection) and the shared-context dictionary pattern. The following table maps each folder to its role in the `main.py` initialization sequence.

Folder	Key Files	Initialized in <code>main.py</code> at Line	Integration Point
<code>config/</code>	<code>settings.py</code> , <code>diagnostics.py</code>	1–23, 66–73	<code>get_settings()</code> provides all configuration dataclasses; <code>run_startup_diagnostics()</code> validates the environment before any module is created
<code>voice/</code>	<code>pipeline.py</code> , <code>vad.py</code> , <code>asr.py</code> , <code>tts.py</code> , <code>face.py</code>	75–76, 81, 108–115, 120	<code>FaceModule</code> and <code>TTSModule</code> are independent; <code>VoicePipeline</code> orchestrates VAD→ASR→LLM→TTS and receives all cross-module references via constructor
<code>vision/</code>	<code>pipeline.py</code> , <code>camera.py</code> , <code>modules/</code>	90, 117–118	<code>VisionPipeline</code> is a standalone thread; <code>get_shared_context()</code> is passed to <code>VoicePipeline</code> as a callable for on-demand vision data injection into LLM prompts
<code>llm/</code>	<code>module.py</code>	82–88, 109	<code>LLMModule</code> owns session lifecycle; <code>create_session()</code> is called before <code>VoicePipeline</code> is created; the instance is injected into <code>VoicePipeline</code>
<code>hardware/</code>	<code>motor_controller.py</code>	92–100, 113	<code>MotorController</code> is conditionally available (graceful degradation); injected into <code>VoicePipeline</code> ; motor commands are triggered by keyword matching in the pipeline worker
<code>shutdown/</code>	<code>main.py</code> (separate FastAPI server)	N/A (separate process)	Deployed independently on Railway; <code>robot_shutdown_client.py</code> (root) polls its API. No direct import from main system.

8. Technical Notes

Overall Tech Stack

Layer	Technology	Purpose
Language	Python 3.10+	All system code
Configuration	<code>python-dotenv</code> + frozen dataclasses	Environment-aware typed settings
Voice VAD	Silero VAD (PyTorch)	Real-time voice activity detection at 16 kHz
Voice ASR	Google Speech Recognition API	Bilingual (Arabic/English) transcription
LLM	OpenRouter API (any model)	Conversational AI with system prompts
Voice TTS	edge-tts (Microsoft Edge)	Natural-sounding bilingual speech synthesis
Face Display	Pygame 2.5+	Animated robot face with state transitions
Vision Camera	OpenCV (cv2.VideoCapture)	Camera capture at configurable resolution/FPS
Vision Face	OpenCV DNN (Caffe SSD)	Face detection + identity tracking
Vision Gesture	MediaPipe Hands	Hand landmark detection → gesture classification
Vision Emotion	Custom CNN (PyTorch)	7-class facial emotion classification
Vision Objects	Ultralytics YOLOv8n	80-class object detection
Vision Scene	Ultralytics YOLOv8n-seg	Instance segmentation for scene understanding
Motor	pySerial → ESP32	Serial protocol (F/B/L/R/SPD commands)
Database	SQLite (via Python sqlite3)	Conversation history per session
Audio I/O	sounddevice (PortAudio)	Low-latency microphone streaming
Fuzzy Matching	rapidfuzz	Wake word and command detection

Design Patterns

Pattern	Location	Description
Dependency Injection	<code>main.py:108-115</code>	<code>VoicePipeline</code> receives all its dependencies (llm, tts, face callback, motor, vision getter) through its constructor rather than importing them directly. This makes the pipeline testable and decoupled.

Pattern	Location	Description
Observer (via Callbacks)	<code>voice/pipeline.py:136-140</code> , <code>voice/face.py</code>	The voice pipeline registers <code>on_start</code> / <code>on_finish</code> callbacks on <code>TTSModule</code> to update the face state. The <code>set_face_state</code> function is passed as a closure from <code>main.py</code> .
Pipeline Architecture	<code>voice/pipeline.py</code> , <code>vision/pipeline.py</code>	Voice: VAD → ASR → Filler → LLM → TTS (sequential stages connected by a thread-safe queue). Vision: Camera → Module Router → per-module processors (parallelizable, with adaptive frame skipping).
Strategy (via Profiles)	<code>config/settings.py:208-232</code>	<code>profile_module_config()</code> returns a dictionary of enabled/disabled flags for each vision module based on the selected <code>VisionProfile</code> enum (MINIMAL/BALANCED/FULL).
Factory Method	<code>config/settings.py:186-197</code>	<code>get_settings()</code> constructs and returns a complete <code>Settings</code> dataclass tree with values from environment variables, acting as a configuration factory.
Graceful Degradation	<code>main.py:97-100</code> , <code>voice/pipeline.py:285-287</code>	Motor controller logs a warning and continues without motors if serial is unavailable. LLM prints a friendly offline fallback message if OpenRouter is unreachable.
Thread-Safe Queue	<code>voice/pipeline.py:151, 180-191</code>	Audio segments flow from the stream thread (callback) to the worker thread via a <code>queue.Queue(maxsize=3)</code> . Dropped-segment counter provides back-pressure visibility.

Threading Model

Thread	Spawned At	Function	Blocking?
Main thread	<code>main.py:124</code>	<code>face.run_main_thread()</code> — Pygame event loop	Yes (blocks indefinitely)
Voice stream	<code>voice/pipeline.py:415</code>	<code>run_forever()</code> — audio capture callback	No (inner sleep loop)

Thread	Spawned At	Function	Blocking?
Voice worker	<code>voice/pipeline.py:413</code>	<code>_worker_loop()</code> — segment processing	No (blocking get with timeout)
Vision pipeline	<code>vision/pipeline.py</code>	Camera read + module routing	No (frame-loop with sleep)
Watchdog	<code>config/diagnostics.py:154</code>	Component health monitoring (daemon)	No (10 s check interval)

All threads are joined with a timeout (3 s for stream, 5 s for worker) during shutdown to prevent hanging.

9. Academic Explanation

System Design Philosophy

Ropo was designed from first principles as a **modular, event-driven robotic system** where each cognitive capability (hearing, seeing, thinking, speaking, moving, expressing) is implemented as an independent module. This follows the **Separation of Concerns** principle: the voice pipeline does not know how vision works, the face module does not know what the LLM said — they communicate through well-defined interfaces (callbacks, context getters, and dependency injection).

The architecture maps naturally to a **sense–think–act** robotic control loop:

- **Sense** — VAD (hearing) + Camera + Vision modules (seeing)
- **Think** — LLM with conversation history and vision context
- **Act** — TTS (speaking) + MotorController (moving) + FaceModule (expressing)

Modularity & Dependency Injection

Every module in Ropo is a Python class that could — in principle — be removed, replaced, or tested in isolation. The key enabler is **constructor-based dependency injection**: `VoicePipeline` receives all its collaborators as parameters. This means:

- A `MockLLM` can replace `LLMModule` for testing without changing a single line of pipeline code
- The `MotorController` gracefully degrades when the ESP32 is not connected — the pipeline simply stops dispatching motor commands
- Vision context is injected lazily via a getter function, so the voice pipeline does not depend on the vision pipeline's import or lifecycle

Bilingual Support

Arabic/English bilingualism is a first-class architectural concern, not a feature bolted on afterwards:

- **ASR** uses `language_mode="auto"` which lets Google's API detect the language per utterance
- **LLM** has separate system prompts for Arabic and English that match the detected language

- **TTS** selects the voice model based on the detected language (`ar-EG-ShakirNeural` for Arabic)
- **Filler phrases** are drawn from separate Arabic/English lists with culturally appropriate expressions
- **Session language** is set at session creation time and stored in the SQLite database
- **Environment documentation** in `.env.example` is written in both languages

Raspberry Pi Optimizations

The system is designed for resource-constrained deployment on Raspberry Pi hardware:

- **Adaptive load shedding** — Vision profiles (minimal/balanced/full) switch module sets at runtime; adaptive frame skipping drops frames when processing exceeds the target interval
- **Platform-specific settings** — Camera resolution defaults to 640×480@15 FPS on RPi vs 1280×720@30 FPS on desktop
- **Thermal monitoring** — CPU temperature is read from `/sys/class/thermal/thermal_zone0/temp`; the system can throttle vision processing at 75 °C
- **Single-threaded Torch/OpenCV** — `configure_platform()` calls `cv2.setNumThreads(1)` and `torch.set_num_threads(1)` to avoid thread contention on limited CPU cores
- **Graceful degradation** — Every component (motor, camera, LLM, microphone) has a runtime-availability check and continues gracefully if unavailable

Graduation Project Value

Ropo demonstrates competency across the full software engineering stack:

- **System Architecture** — Multi-threaded pipeline design with clear separation of concerns, dependency injection, and callback-based communication
- **Computer Vision** — Integration of four distinct vision techniques (DNN-based face detection, MediaPipe hands, YOLO object detection, CNN emotion classification) into a unified pipeline
- **Natural Language Processing** — Bilingual speech recognition, LLM-based conversation with context management, and natural TTS synthesis
- **Embedded Systems** — Serial communication with ESP32 for motor control, GPIO-aware systemd service for safe shutdown
- **DevOps & Tooling** — Cross-platform setup scripts (bash + PowerShell), automated diagnostics with readiness scoring, logging with rotation, and environment-based configuration

- **Software Quality** — Comprehensive error handling (every exception is caught at the appropriate level), graceful degradation, thread-safe queue communication, and watchdog monitoring

The project is **fully functional on Windows desktop** and **deployment-ready for Raspberry Pi 5**, making it a complete end-to-end system that bridges academic AI/ML knowledge with practical robotics engineering.

Ropo Educational Robot — System Documentation

Generated from source code analysis. All code references are to the project root at the time of writing.